

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Design an online examination website for lecturers
and students**

Đoàn Tuấn Hùng

hung.dt205154@sis.hust.edu.vn

Major: Information Technology

Program: Global ICT

Supervisor: Assoc. Prof. Truong Thi Dieu Linh

Signature

Department: Computer Engineering

School: Information and Communications Technology

HANOI, 06/2025

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to all those who have contributed to the successful completion of my thesis.

Above all, I extend my heartfelt thanks to my supervisor, Mrs. Truong Thi Dieu Linh, for her invaluable guidance, unwavering support, and encouragement throughout my thesis. Her extensive knowledge has played a crucial role in shaping my thesis and assisting me in overcoming challenges.

I would also like to acknowledge the efforts and contributions of individuals who have provided support and insights along the way. This accomplishment would not have been possible without the encouragement and assistance of those who have supported me during this journey. I am truly thankful for the collective efforts that have played a significant role in the successful completion of my thesis.

ABSTRACT

The growing shift toward digital education has exposed the limitations of current online examination tools, particularly for small to mid-sized educational institutions. Many existing platforms, such as Google Forms or commercial learning management systems, are either too complex or lack essential features tailored for exam administration. Teachers often face difficulties in assigning exams to specific classes, managing question banks, and tracking student performance efficiently. Similarly, students struggle with accessing scheduled exams, viewing rankings, or even understanding when an exam is available. These gaps highlight the need for a focused and lightweight online exam system designed specifically for educational use cases.

This thesis adopts a minimalist, modular approach to address these challenges, aiming to deliver a practical web-based platform without heavy infrastructure or third-party dependencies. The chosen solution uses a PHP and MySQL backend to handle user roles, exam schedules, class assignments, and performance data. The frontend is implemented using standard web technologies — HTML, CSS, and JavaScript — with Bootstrap for responsive design. A notable feature is the ability to import student data via CSV using native PHP file handling, avoiding reliance on external spreadsheet libraries. Exams are made visible to students dynamically based on scheduled timestamps stored in the database.

The implemented system provides separate, streamlined interfaces for teachers and students. Teachers can create and assign exams to specific classes, manage questions centrally, and monitor class-specific performance rankings. Students, in turn, can view and take available exams, see their own rankings, enabling better self-assessment.

The main contribution of this thesis is a robust exam management platform that meets the real-world needs of institutions with limited technical resources. The final system provides an effective foundation for future development toward a comprehensive online examination environment.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and Scope of the Thesis	1
1.3 Tentative Solution.....	2
1.4 Thesis organization.....	3
CHAPTER 2. TECHNOLOGY STACK AND APPROACH.....	4
2.1 Frontend technologies	4
2.1.1 HTML (HyperText Markup Language)	4
2.1.2 CSS (Cascading Style Sheets)	4
2.1.3 JavaScript.....	4
2.1.4 Bootstrap.....	5
2.2 Backend technologies.....	5
2.2.1 PHP (Hypertext Preprocessor)	5
2.2.2 MySQL.....	5
CHAPTER 3. SYSTEM DESIGN AND ANALYSIS.....	6
3.1 Status survey	6
3.1.1 Moodle	6
3.1.2 Google Classroom.....	7
3.2 Functional overview.....	9
3.2.1 General use case diagram.....	9
3.2.2 Detailed use case diagram.....	9
3.3 Functional description.....	16
3.3.1 Sign up use case.....	16
3.3.2 Login use case	17

3.3.3 Take exam use case	18
3.3.4 Directly add exam question use case	19
3.3.5 Add question bank question use case.....	20
3.4 Architectural design.....	21
3.5 Overall design	22
3.5.1 Frontend.....	22
3.5.2 Backend	23
3.6 User Interface design	23
3.6.1 Sidebar navigation.....	24
3.6.2 Main content area	24
3.6.3 Buttons	25
3.6.4 Responsiveness and Role-Driven Adaptability	25
3.7 Database design.....	26
3.7.1 Entity-relationship diagram.....	26
3.7.2 Database schema.....	27
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	30
4.1 Achieved results	30
4.1.1 Student's interfaces	32
4.1.2 Lecturer's interfaces	35
4.1.3 Testing	37
CHAPTER 5. CONCLUSION AND FUTURE WORK	40
5.1 Conclusion.....	40
5.2 Future work.....	41
APPENDIX	43

LIST OF FIGURES

Figure 3.1	Moodle Interface	7
Figure 3.2	Google Classroom interface	8
Figure 3.3	General use case diagram	9
Figure 3.4	View exam diagram	10
Figure 3.5	Take exam diagram	10
Figure 3.6	See ranking diagram	11
Figure 3.7	View history diagram	12
Figure 3.8	Manage exam diagram	13
Figure 3.9	Manage student list diagram	14
Figure 3.10	Manage question bank diagram	14
Figure 3.11	Architecture diagram	21
Figure 3.12	Frontend architecture diagram	22
Figure 3.13	Backend architecture diagram	23
Figure 3.14	Sidebar screen	24
Figure 3.15	Main content screen	25
Figure 3.16	Responsive interface	26
Figure 3.17	Entity-relationship diagram	26
Figure 3.18	Database schema	27
Figure 4.1	Exam website role interface	30
Figure 4.2	Login interface for students	31
Figure 4.3	Registration interface for students	32
Figure 4.4	Home page interface for students	32
Figure 4.5	Take exam interface for students	33
Figure 4.6	History interface	33
Figure 4.7	History detail interface	34
Figure 4.8	Ranking interface	34
Figure 4.9	Notification interface	35
Figure 4.10	Home page interface for Lecturers	35
Figure 4.11	Manage exam interface	36
Figure 4.12	Question bank interface	36
Figure 4.13	Lecturer notification interface	37

LIST OF TABLES

Bảng 3.1	Signup use case description	16
Bảng 3.2	Login use case description	17
Bảng 3.3	Take exam use case description	18
Bảng 3.4	Directly add exam question use case description	19
Bảng 3.5	Add question to bank use case description	20
Bảng 3.6	Detail of the entity "User"	27
Bảng 3.7	Detail of the entity "Class"	27
Bảng 3.8	Detail of the entity "Quiz"	28
Bảng 3.9	Detail of the entity "History"	28
Bảng 3.10	Detail of the entity "History_Details"	28
Bảng 3.11	Detail of the entity "Exam_Questions"	28
Bảng 3.12	Detail of the entity "Question bank"	29
Bảng 3.13	Detail of the entity "Options"	29
Bảng 4.1	Add question into bank Test case	37
Bảng 4.2	Add student into class Test case	38
Bảng 4.3	Add student into class Test case	39

LIST OF ABBREVIATIONS

Abriviation	Full Expression
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHAPTER 1. INTRODUCTION

1.1 Motivation

With the increasing shift toward digital education, the need for effective on-line examination systems has become more urgent. Many educational institutions — especially small to mid-sized schools — face difficulties in managing online exams due to the limitations of current tools. Existing solutions such as Google Forms, or commercial LMS platforms either offer too many complex features unrelated to exams or lack critical exam-centric functionalities such as class-based exam assignment, question bank reuse, and simple ranking visibility.

Without a specialised online exam system, teachers often struggle to assign exams to specific student groups, track performance in a structured way, and reuse or manage questions without repetitive input. On the student side, accessing scheduled exams, viewing performance rankings, or even confirming whether an exam is currently open is often unclear or not supported at all.

There is a clear need for a streamlined, purpose-built examination platform that addresses these specific gaps — a system that supports exam creation, class assignment, ranking visibility, question management, and bulk data handling without unnecessary dependencies or complexity.

1.2 Objectives and Scope of the Thesis

The objective of this thesis is to develop a lightweight, modular web application for managing online exams that implement essential functions of the online exam system. Existing systems often suffer from excessive complexity, reliance on external dependencies. This thesis aims to solve those issues by proposing a streamlined, role-based exam system that can operate without heavy infrastructure or commercial libraries, focusing on class-based exam assignment and visibility control.

The system provides a range of functionalities for teachers, who are the primary administrators of the examination process. Teachers will be able to log into the platform to create exams, assign them to specific student classes, and control when exams become visible using flexible timestamp logic. They can also manage a centralized question bank, adding or removing questions as needed. To ease student management, the platform will allow bulk student assignments to classes through simple CSV uploads, eliminating the need for third-party spreadsheet libraries. Teachers will also have access to student lists and can monitor class-specific per-

formance rankings, improving oversight and transparency.

For students, the system offers a focused and minimal interface tailored to their needs. Students will be able to log in and view a list of currently available or upcoming exams based on their class and the exam's active status. The platform will also allow students to take exam, see their performance rankings, enabling self-assessment and motivation.

This project targets small educational institutions or schools seeking a simple, effective, and low-maintenance platform for exam management. By avoiding unnecessary features and heavy dependencies, the system ensures smooth operation even in constrained environments. Its modular design and clearly defined user roles also make it easier to maintain and expand in the future, establishing a practical foundation for ongoing development.

1.3 Tentative Solution

By addressing current system limitations identified in Section 1.1, the solution seeks to improve how digital exams are conducted in classroom environments.

The system's web-based architecture ensures broad accessibility and ease of use: it eliminates installation overhead, allowing both teachers and students to interact with the system using any internet-connected device. Without third-party dependencies, the system remains lightweight and portable, making it well-suited for offline deployment or constrained environments with limited internet access or technical support.

The technical foundation of the system is built using PHP for backend logic and MySQL for managing relational data structures such as users, exams, classes, and questions. The user interface will be developed with HTML, CSS, and JavaScript, with Bootstrap providing responsive design elements to ensure compatibility across devices. CSV import functionality will be implemented using PHP's native file handling features to simplify student enrollment without relying on third-party libraries such as PhpSpreadsheet. Exam access will be managed using a combination of start time and end time in the database, allowing dynamic control over when exams become available to students based on system time.

The expected outcome of this project is a complete, functional exam management platform that does not rely on external software dependencies, making it suitable for lightweight and offline-friendly deployments. The system's modularity will also support future expansion into features like online exam-taking, automated grading, and performance analytics. Ultimately, this project contributes a practical

tool for enhancing digital assessment practices, especially in educational environments with limited technical infrastructure.

1.4 Thesis organization

Chapter 2 will describe the technologies required in the progress of designing the online examination system.

After giving an overview of the technologies required, chapter 3 will shift the focus to the design and analytical aspects of the system. It includes basic design of the exam system such as architecture, database,.....

Chapter 4 will demonstrate the testing results of the result of the design and development process of the system, including its basic functions as well as the test cases

Finally, chapter 5 will give the conclusion to the development process of the exam system, explaining what had been achieved and what limitations the website had, and propose future work to enhance its functionalities.

CHAPTER 2. TECHNOLOGY STACK AND APPROACH

With the requirements outlined in the previous chapters, I have chosen a lightweight and widely supported set of technologies to develop and implement the exam management web application. The chosen stack includes PHP and MySQL for the backend, and HTML, CSS, JavaScript, and Bootstrap for the frontend. This stack was selected due to its simplicity, broad compatibility, minimal dependencies, and ease of deployment, particularly in environments where server resources may be limited or where external libraries (such as Node.js or Python packages) are impractical. This chapter provides a brief overview of the technologies used and the reasoning behind their selection.

2.1 Frontend technologies

The frontend of the system is responsible for the user interface and the user experience of both students and teachers. It is built using standard web technologies including HTML, CSS, and JavaScript, along with the Bootstrap framework.

2.1.1 HTML (HyperText Markup Language)

HTML is the standard markup language used to define the structure and content of web pages. In this exam system, HTML forms the foundation of every screen—from login pages and exam listings to student profiles and teacher dashboards. All interface components such as tables, input forms, modals, buttons, and navigation elements are structured using semantic HTML tags, which ensures accessibility, clarity, and compatibility across browsers.

2.1.2 CSS (Cascading Style Sheets)

CSS is used to style and visually present the HTML structure defined across the application. Through CSS, the system achieves consistent branding, layout control, color schemes, typography, and spacing. In the exam application, CSS enhances user experience by ensuring that key elements such as exam cards, form fields, buttons, and class listings are visually distinct and intuitively styled, making the interface more appealing and user-friendly, especially for mobile or tablet use.

2.1.3 JavaScript

JavaScript is a client-side scripting language used to introduce interactivity and dynamic behavior into web pages. In the exam system, JavaScript is used to handle asynchronous operations such as form submission, modal toggling, form validation, and table filtering. These dynamic behaviors allow for a smoother and more responsive experience without requiring full page reloads. For instance, when

teachers upload a CSV file to add students in bulk, JavaScript captures the file input and communicates with the backend in the background, enhancing usability.

2.1.4 Bootstrap

Bootstrap is a popular frontend framework that provides pre-built CSS classes and JavaScript components for responsive web design. In this system, Bootstrap significantly speeds up interface development and ensures mobile responsiveness. It is used to create modals, buttons, form layouts, navigation bars, and tables with a consistent and modern look. Its responsive grid system allows the application to work effectively across a range of devices, which is essential for students accessing exams on phones or tablets.

2.2 Backend technologies

The backend of the system handles data processing, business logic, and server-side operations. The system uses PHP as the primary server-side scripting language and MySQL as the database management system.

2.2.1 PHP (Hypertext Preprocessor)

PHP is a widely-used, open-source scripting language designed for server-side development. In this exam system, PHP handles core logic such as authentication, exam scheduling, question management, class creation, and CSV import. PHP scripts respond to frontend requests by interacting with the database and generating appropriate responses. The choice of PHP is influenced by its lightweight nature, extensive documentation, and compatibility with shared hosting environments, making the system easy to deploy and maintain, especially for institutions with limited resources.

2.2.2 MySQL

MySQL is a relational database management system used to store and organize all data in the exam application, including user accounts, questions, class information, exams, and student rankings. The system uses structured tables with relational mappings to efficiently retrieve, update, and manage data. SQL queries executed through PHP enable dynamic content generation and data validation. MySQL was chosen for its robustness, support for ACID-compliant transactions, and ability to handle complex queries, which are necessary for features like generating class-wise performance rankings or filtering exams based on time windows.

CHAPTER 3. SYSTEM DESIGN AND ANALYSIS

Chapter 2 has introduced technologies used to solve the system requirements. Chapter 3 will shift the focus to the design and analytical aspects of the system. It includes detailed examination of system requirements as well as class design of the system. Afterwards, the UI design and database design will be presented.

3.1 Status survey

To accomplish the goals discussed in the previous chapter regarding the design and development of the online exam system, it is important to review popular existing platforms to identify essential features and functionalities. In this project, I will analyze two systems : Moodle and Google Classroom

3.1.1 Moodle

Moodle is a widely-used, open-source Learning Management System (LMS) [1] that offers a rich set of tools for managing online education, including quizzes and exams. For teachers, Moodle provides powerful features such as creating and assigning quizzes to classes, viewing detailed exam results, adding questions directly or from a centralized question bank, and managing student enrollments. Teachers also have access to detailed analytics and ranking systems to evaluate student performance over time.

For students, Moodle allows secure login, access to assigned quizzes or exams, and a personalized dashboard for navigating between classes and activities. Students can view their scores and feedback after taking exams, fulfilling functionalities like “View Exam” and “See Ranking.” Moodle also supports a robust question bank that teachers can use to reuse or organize questions into categories, making the system scalable and efficient.

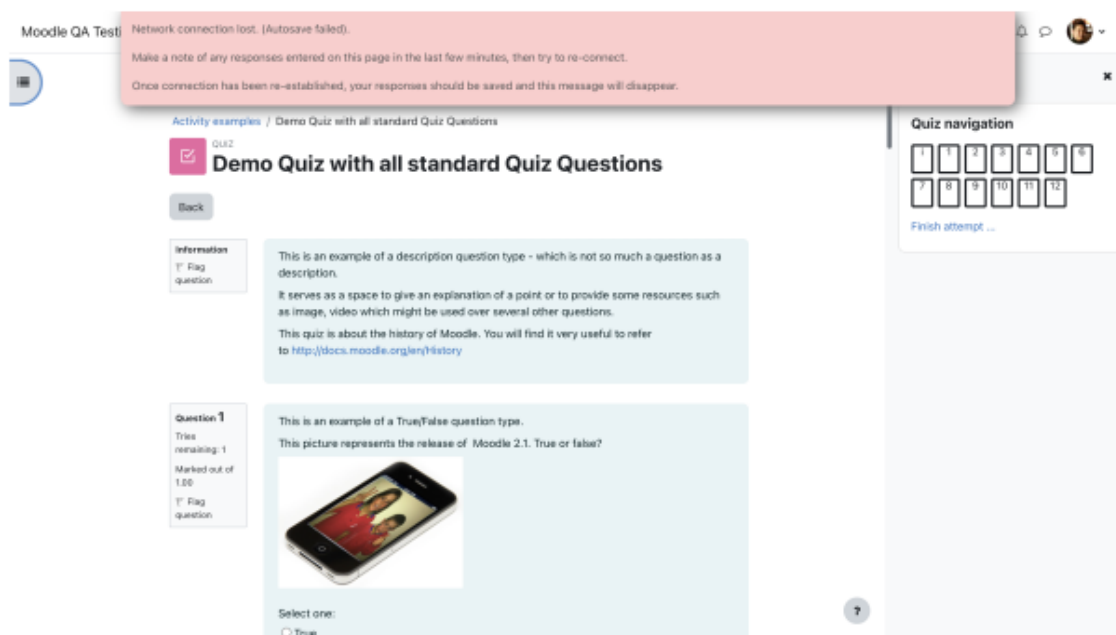


Figure 3.1: Moodle Interface

3.1.2 Google Classroom

Google Classroom [2] is a free online learning tool developed by Google to streamline the process of sharing files, assignments, and feedback between teachers and students. While not a dedicated exam system, it supports quizzes through integration with Google Forms, making it a simple but functional option for online assessments. Teachers can create classes, invite students, post announcements, distribute exams via Google Forms, and view scores, create question sets using Forms, which can be reused and automatically graded.

For students, Google Classroom provides an easy-to-navigate interface where they can log in, view assignments (including exams), and submit their work. Integration with Google Sheets can provide basic analytics or rankings.

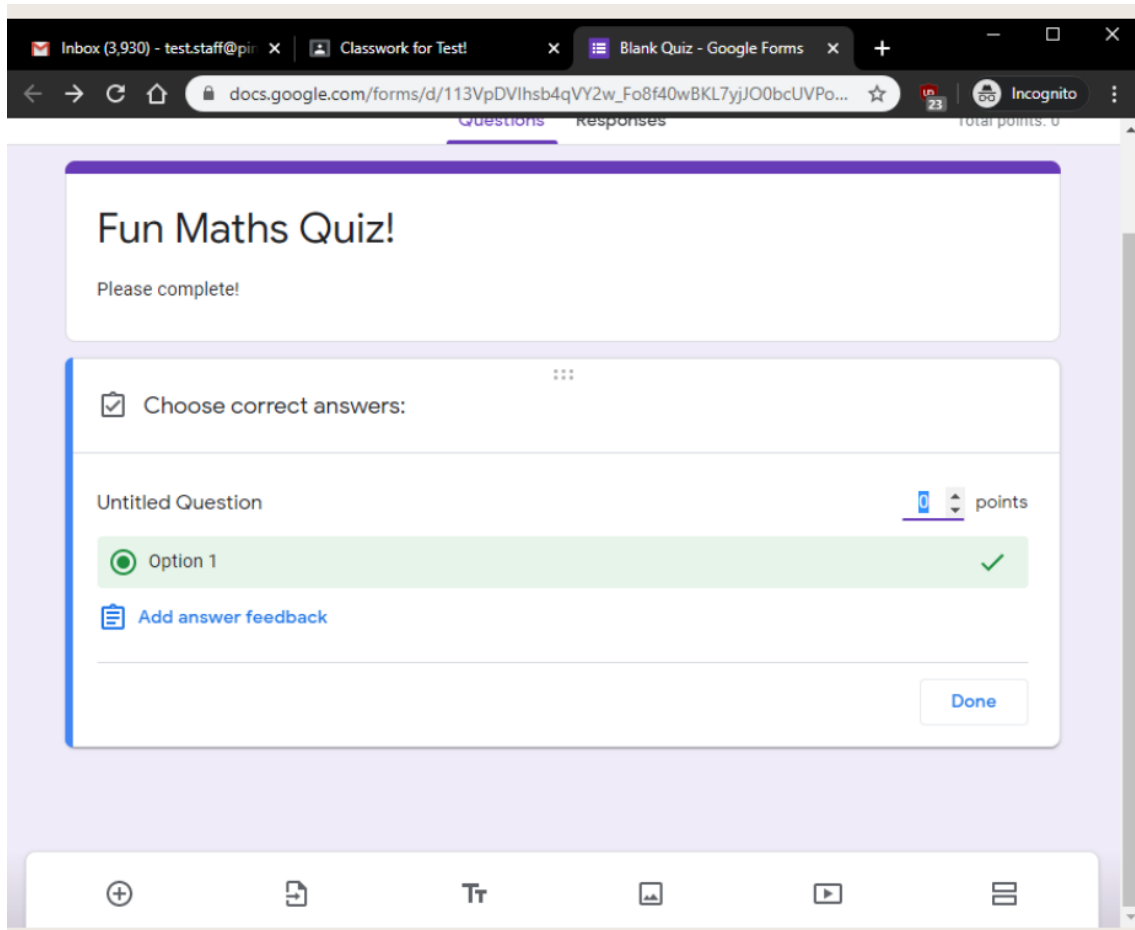


Figure 3.2: Google Classroom interface

After surveying Moodle and Google Classroom, the following essential features for an effective online exam system are identified:

Secure and Simple Authentication : Easy and safe login for students and teachers.

Exam Management: Add/edit/remove exams and view detailed metadata, assign exams to specific classes.

Student Interaction Tools: Taking exams, viewing rankings, and interacting with question banks.

Advanced Filtering and Analytics: Filtering exams by class or subject, providing ranking reports and exam history.

Question Bank Integration: Adding questions directly or from a centralized, reusable question pool.

Class and User Management: Adding/removing students and assigning them to classes.

User Roles with Permissions: Separation of views and permissions between

teachers and students.

3.2 Functional overview

3.2.1 General use case diagram

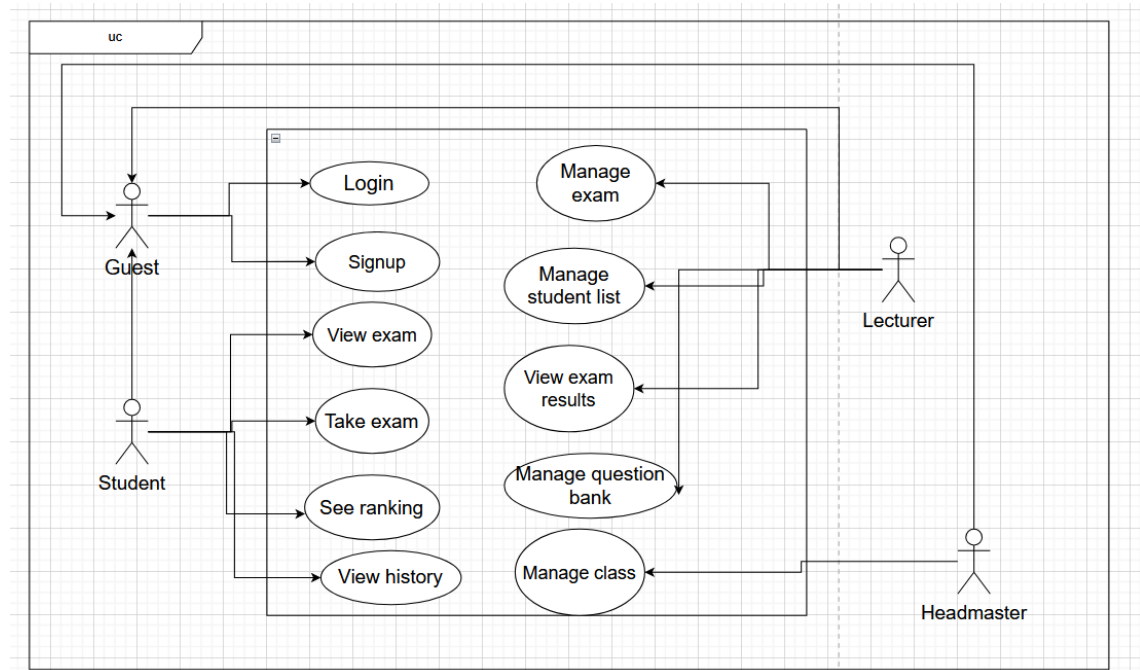


Figure 3.3: General use case diagram

As shown in the Use Case Diagram above, we have the following actors : Guest (Users who haven't logged in), Students, Lecturers and Headmasters.

Headmaster is the highest-level actor, including use cases such as managing classes, give feedbacks.

Guest : Users who can enter the website without logging in. They can't access the website functionalities as a result. Guest can sign up for a Lecturer account or a Student account and log in to the system using the created account

Students : Students are users who have logged in the system with their student account. They can use functionalities such as view exam, take exam, and they can also view history and see the rankings

Lecturers: Lecturers are users who have logged in the system with their lecturer accounts. They can perform activities including managing exam, student list, question bank and viewing exam results

3.2.2 Detailed use case diagram

Detailed use case diagram "View exam"

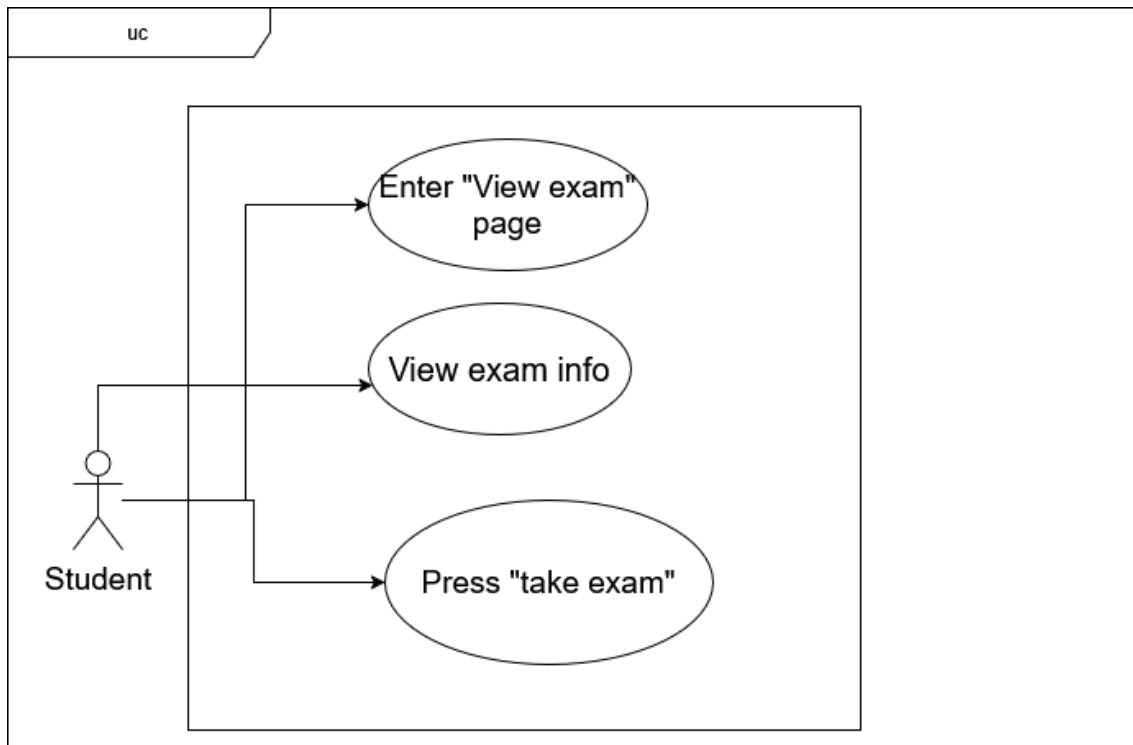


Figure 3.4: View exam diagram

Initially, the students access the exams available in the system, depending on the class they're currently in. After searching for the desired exam, the user proceeds to take the exam. The system asks for confirmation, and if students click yes, the system will transfer to the "Take exam" interface.

Detailed use case diagram "Take exam"

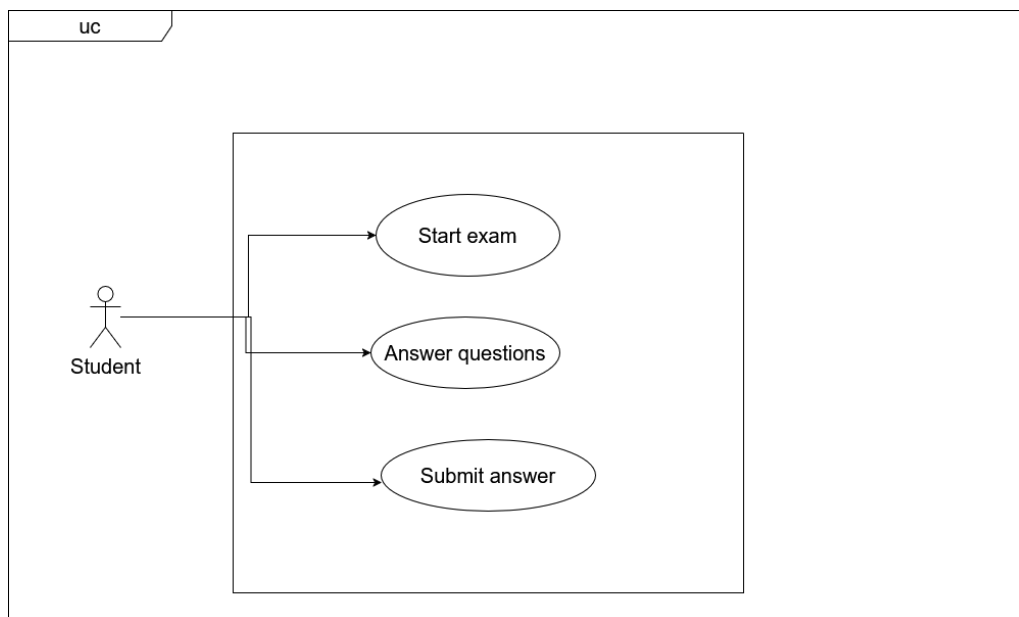


Figure 3.5: Take exam diagram

After taking exam, the students will have to answer the questions in the exam in

limited time, depending on the exam's time limit. Students can use the question button to navigate to the current question, or check if question is answered or not. After answering all the questions, students click the Submit button and confirm submitting the exam.

Detailed use case diagram "See ranking"

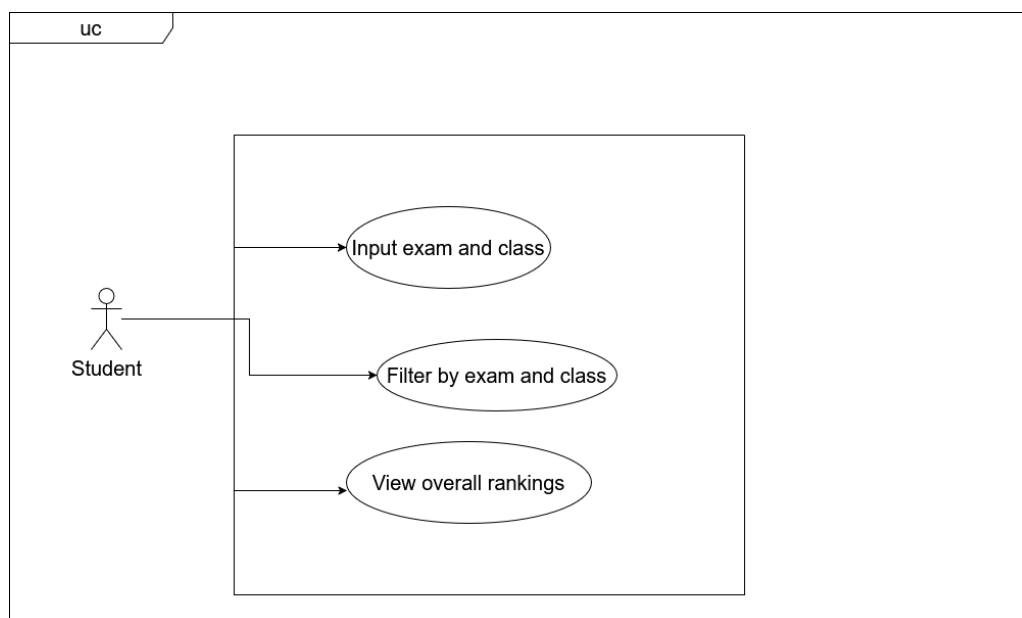


Figure 3.6: See ranking diagram

After taking the exam, students can access the ranking table to see their final score. They first need to input the exam title and class name, and submit the input result. A ranking table will be shown, showing their exam score and their position

Detailed use case diagram "View history"

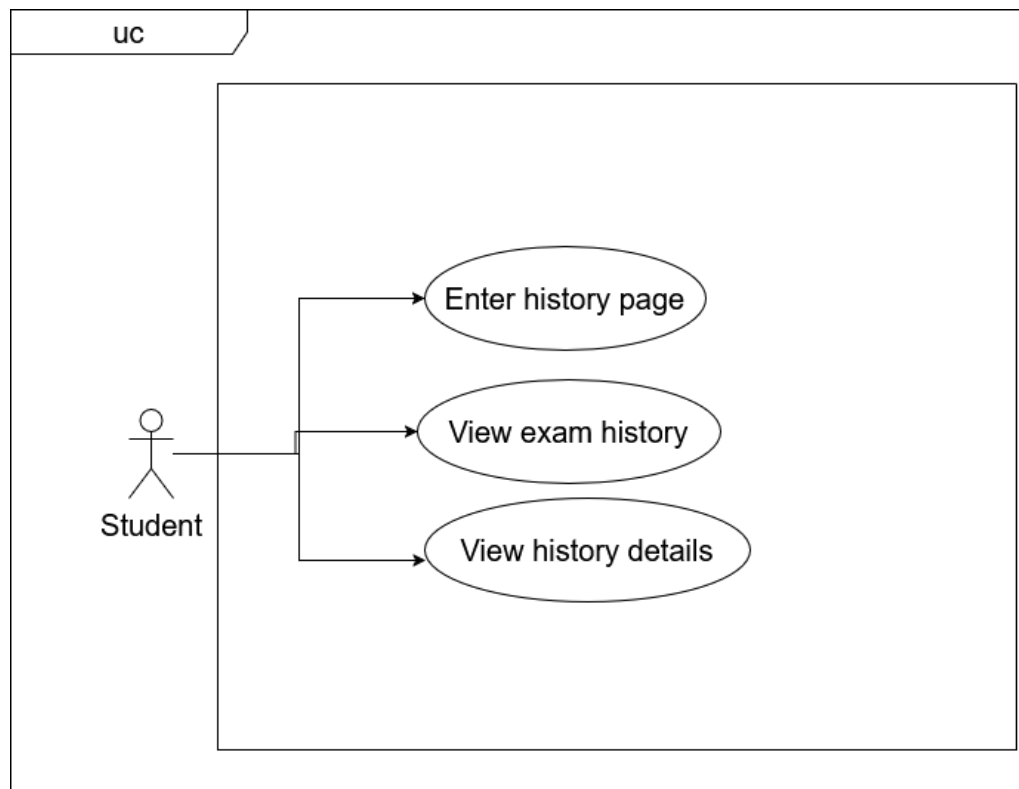


Figure 3.7: View history diagram

Upon submitting the exam, students will be directed to the History page, where they can view the history of exams they took. Upon clicking Details button, users can view details of the exam they had taken, including questions answered, correct answer,.....

Detailed use case diagram "Manage exam"

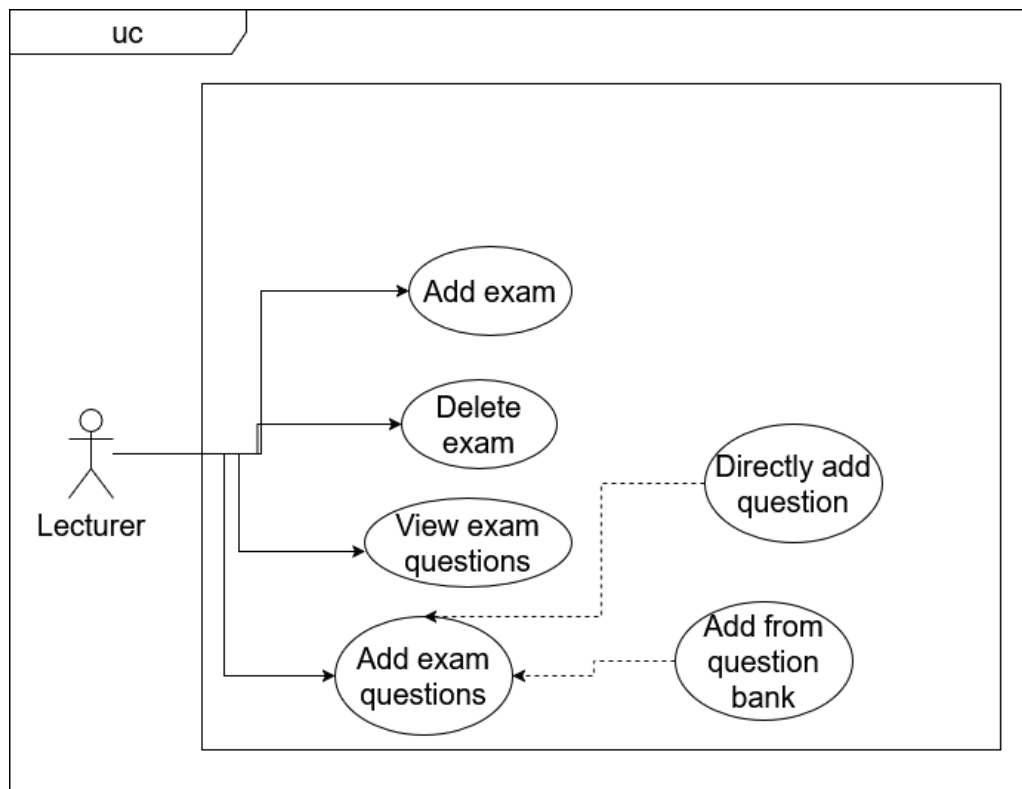


Figure 3.8: Manage exam diagram

Lecturers upon entering Manage exam, are able to see the list of exams they created. They can create new exams, and for each exams, they can see the question details, add exam questions or delete exams. After lecturers proceed to add exam question, they have options to add exam question through the question bank, or directly add exam question.

Detailed use case diagram "Manage student list"

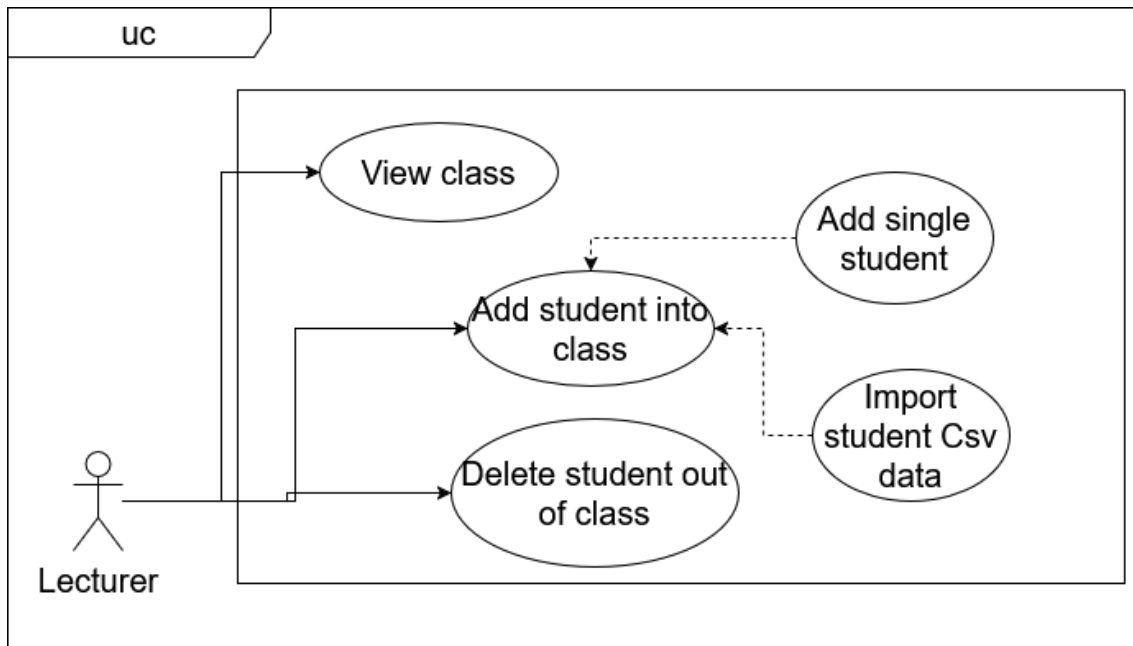


Figure 3.9: Manage student list diagram

After logging in, lecturers can access the student list page, where they can add/remove students out of class. When adding a student, they had 2 options: either by adding one single student, or by adding multiple students via Csv file.

Detailed use case diagram "Manage question bank"

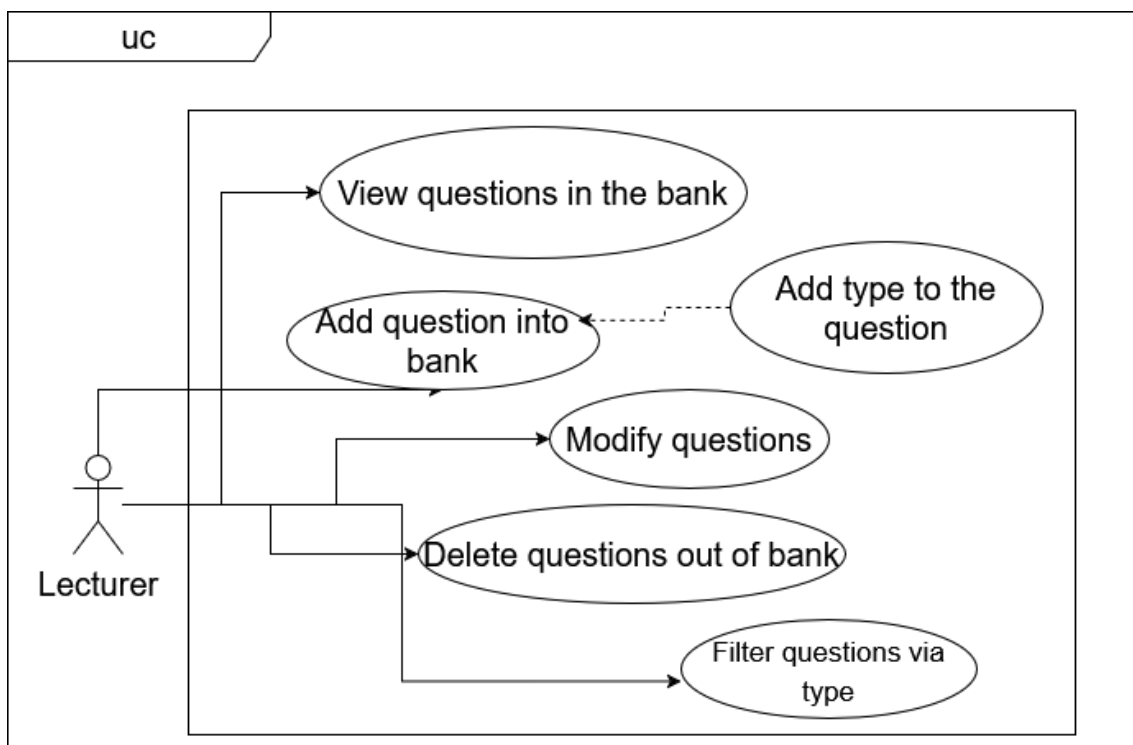


Figure 3.10: Manage question bank diagram

Upon entering the "Question bank" page, lecturers will be able to view all ques-

tions inside the bank. They can add a new question and filter the questions in the bank through types, and for each questions in the bank, they can either modify, delete question, or assign multiple types to one question.

3.3 Functional description

3.3.1 Sign up use case

Usecase code	U001
Usecase name	Sign up
Actor	Guest
Description	Describe the process that guests create a student, lecturer or headmaster account on the website
Pre-condition	Guest did not log in

Main flow:

No.	Performed by	Action
1	Guest	Navigate to the exam website login page
2	System	Display role box, asking for guests to pick role
3	Guest	Click on the desired role
4	System	Displays login page depend on role
5	Guest	Click on "Create account"
6	System	Displays signup page depend on role
7	Guest	Fill the information and click "Sign up"
8	System	Validate that the information is complete and check its uniqueness
9	System	Notice signup successfully

Alternative flow:

No.	Performed by	Action
8.1	System	Check invalid information, notice and ask guest refill
8.2	System	Check account exists, ask guest to use other username
8.3	System	Check if headmaster account exists, ask guest to signup for other role

Post-condition: No

** The input data consists of the following data fields:*

No.	Data field	Required?	Short description
1	Username	Yes	hungx16
2	Real Name	Yes	Doan Tuan Hung
3	Password	Yes	12345
4	Re-password	Yes	12345

Table 3.1: Signup use case description

3.3.2 Login use case

Usecase code	U002
Usecase name	Login
Actor	Guest
Description	Describe the process that guests picked the role and login to the exam website
Pre-condition	Guest had account

Main flow:

No.	Performed by	Action
1	Guest	Navigate to the exam website login page
2	System	Display role box, asking for guests to pick role
3	Guest	Click on the desired role
4	System	Displays login page depend on role
5	Guest	Input information and click "Login"
6	System	Check if any information guest inputted is correct
7	System	Directs to the Home page depend on account's role guest logged in

Alternative flow:

No.	Performed by	Action
6.1	System	Check invalid information, notice and ask guests to refill
6.2	System	Check if the role user attempt to login correspond with the role of the account user inputted
6.2	System	Notifies "Unauthorised access"

Post-condition: No

** The input data consists of the following data fields:*

No.	Data field	Required?	Short description
1	Username	Yes	hungx16
2	Password	Yes	12345

Table 3.2: Login use case description

3.3.3 Take exam use case

Usecase code	U003
Usecase name	Take exam
Actor	Student
Description	Describe process that student takes exam
Pre-condition	Student is logged in and already added to a class by lecturer

Main flow:

No.	Performed by	Action
1	Student	Navigate to the exam website home page
2	System	Display exam list depend on the class the student belongs to
3	Student	Click on the "Take" button, depend on the exam they want to take
4	System	Confirms the student choice
5	System	Displays exam with questions and timer
6	Student	Answer all questions and click "Submit"
7	System	Confirms submission
8	System	Directs to history page to see results

Alternative flow:

No.	Performed by	Action
2.1	System	Check exam's toggle status, stop students from taking the exam
7.1	System	Check if student ignored exam question or not, ask student to answer ignored question
6.1	System	Check if student runs out of time during exam
6.2	System	Notify "Time's up" and immediately jump to step 8

Post-condition: No

** The input data consists of the following data fields:*

No.	Data field	Required?	Short description
1	Username	Yes	hungx16
2	Exam id	Yes	1
3	Exam question id	Yes	1
4	Answer chosen	Yes	A,B,C,D,...
5	Answer id	Yes	1
6	Time	Yes	2025-05-06 13:39:03

Table 3.3: Take exam use case description

3.3.4 Directly add exam question use case

Usecase code	U004
Usecase name	Directly add exam question
Actor	Lecturer
Description	Describe the process that lecturers directly add question to an exam
Pre-condition	Lecturer logged in

Main flow:

No.	Performed by	Action
1	Lecturer	Enter the "Manage Exam" page
2	System	Display list of exams
3	Lecturer	Click "Add exam question"
4	System	Displays modal asking user to choose method of adding question
5	Lecturer	Click on "Directly add question"
6	System	Shows modal with input fields
7	Lecturer	Fill out all necessary info and click on "Add question"
8	System	Validates input
9	System	Inserts question directly into respective exam (using eid)
10	System	Notifies "question added successfully"

Alternative flow:

No.	Performed by	Action
4.1	System	If any required field is empty or invalid, display validation message and return to form
4.2	System	Corrects the input and resubmits
4.3	System	Proceed to step 5 at main flow

Post-condition: No

** The input data consists of the following data fields:*

No.	Data field	Required?	Short description
1	Question title	Yes	1+1=?
2	Number of choice	Yes	2-6
3	choice A	Yes	Two
4	choice B	Yes	Three
5	choice C to D	Conditional	Extra options
6	Correct answer	Yes	A-F
7	Question type	yes	Math
8	Eid	Yes	1

Table 3.4: Directly add exam question use case description

3.3.5 Add question bank question use case

Usecase code	U004
Usecase name	Add exam question
Actor	Lecturer
Description	Describe the process that lecturers add exam question to the question bank
Pre-condition	Lecturer logged in

Main flow:

No.	Performed by	Action
1	Lecturer	Enter the "Question bank" page
2	System	Display list of questions in the question bank
3	Lecturer	Click "Add exam question"
4	System	Displays modal asking user to input question info
5	Lecturer	Click on "Add question"
6	System	Notify "Question added successfully"

Alternative flow:

No.	Performed by	Action
4.1	System	Check empty or invalid question info, or if duplicate question already exists in the question bank
4.2	System	Notice and ask lecturer to input again

Post-condition: No

** The input data consists of the following data fields:*

No.	Data field	Required?	Short description
1	Question title	Yes	1+1=?
2	Number of choice	Yes	2-6
3	choice A	Yes	Two
4	choice B	Yes	Three
5	choice C to D	Conditional	Extra options
6	Correct answer	Yes	A-F
7	Question type	yes	Math
8	Score	yes	2
9	Eid	Yes	1

Table 3.5: Add question to bank use case description

3.4 Architectural design

The software architecture of this system adopts the Three-Layer Architecture, which is a well-established software design pattern. This architecture is composed of three primary layers: the Presentation Layer, Application Layer, and Data Access Layer.

Figure 3.11 provides an illustration of the system's architecture.

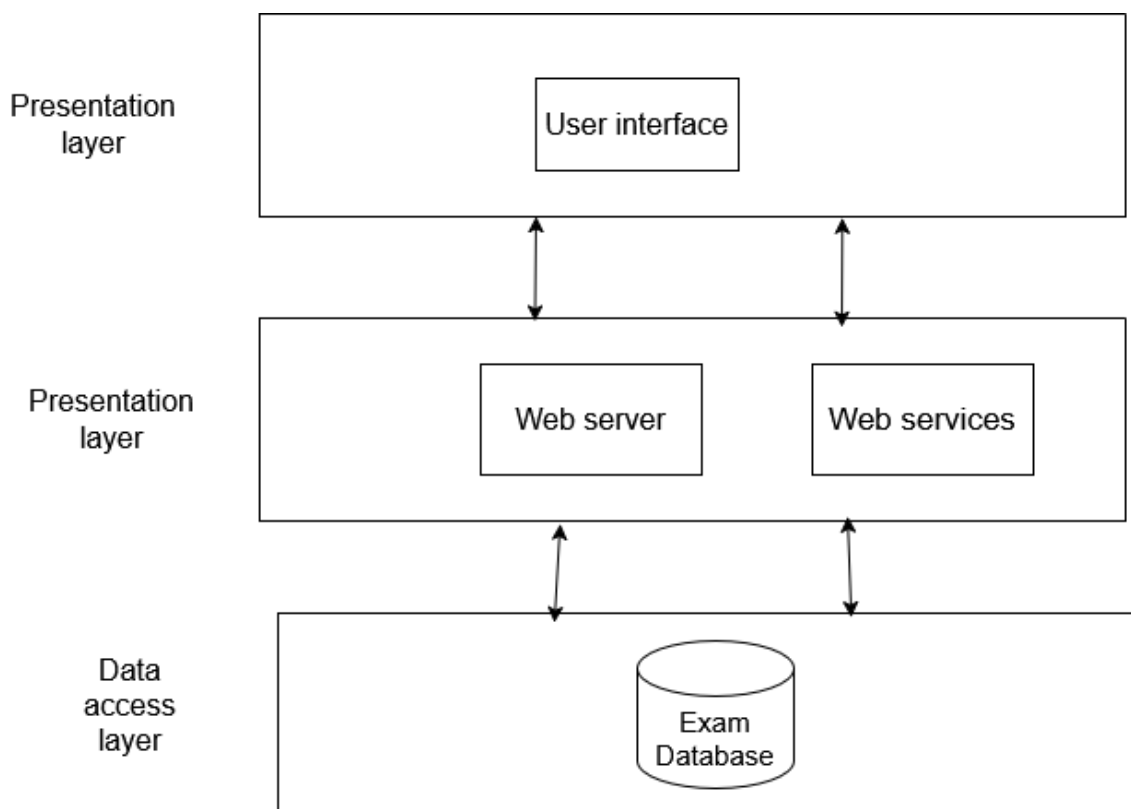


Figure 3.11: Architecture diagram

The Three-Layer Architecture organizes the application into three logical layers that can also be deployed across separate physical tiers:

Presentation layer: The presentation layer acts as the user-facing interface of the exam system, where students, teachers, and administrators interact with the application. Its main responsibilities include rendering exam data, capturing user input, and providing feedback or navigation based on actions taken. This layer is built using HTML, CSS, and JavaScript, with Bootstrap enhancing responsiveness and visual consistency across various screen sizes. The primary goal of this layer is to ensure a user-friendly, accessible interface that requires no special installation and works across devices.

Application layer: The application layer, implemented using PHP, is the core processing engine of the system. It manages all logical operations, such as vali-

dating user credentials, controlling exam visibility based on time,..... This layer bridges the frontend and backend, responding to HTTP requests sent by the presentation layer, and making necessary decisions before querying or updating the database. The focus here is to keep the logic modular, secure, and easily extensible to accommodate future features like analytics and auto-grading.

Data access layer: The data access layer is responsible for storing, retrieving, and managing application data. This is implemented using MySQL, a relational database system chosen for its reliability and structured querying capabilities. PHP scripts from the application layer execute SQL queries to perform major operations, ensure data consistency, and support features like filtering by class, aggregating performance, and recording exam history. This layer is fundamental in maintaining data integrity and system stability, especially under concurrent usage scenarios.

3.5 Overall design

3.5.1 Frontend

Figure 3.12 shows the frontend architecture:

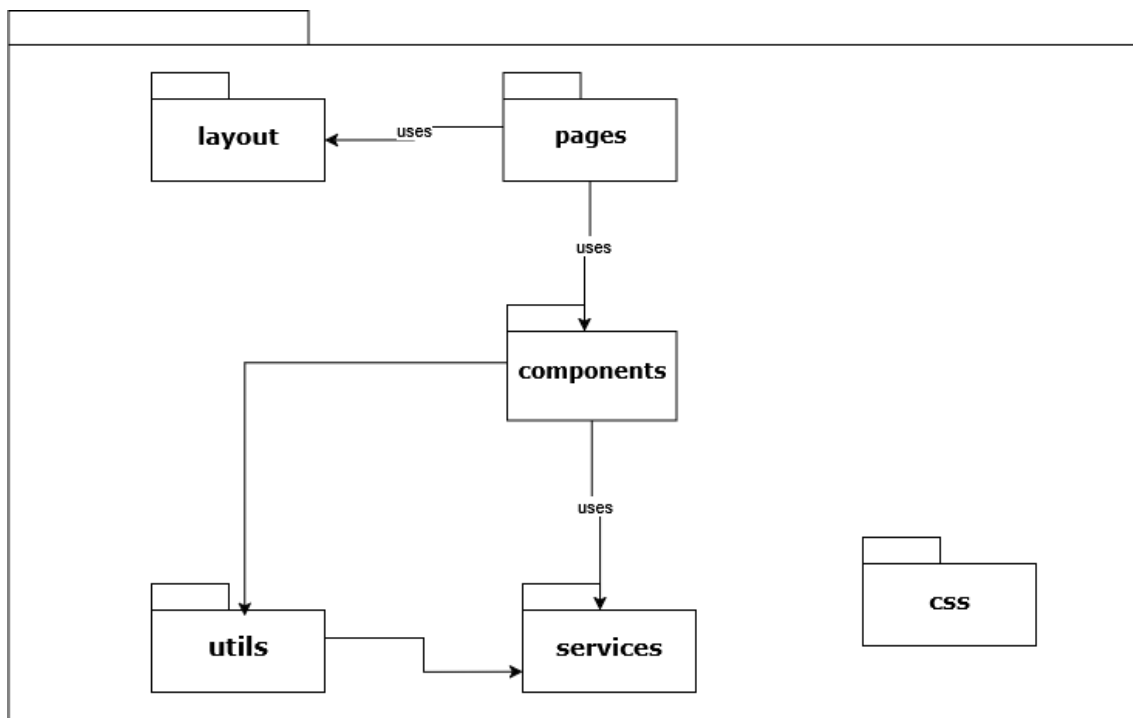


Figure 3.12: Frontend architecture diagram

The **pages** folder serve as the entry points for each user-facing screen.

The **components** folder includes reusable PHP-HTML blocks like: exam table rows, alert boxes, modal dialogs,.....

The **layouts** folder contains layout components like sidebar.php, navbar.php, or

footer.php.

The **service** folder include PHP logic helpers, e.g.: DB queries, session/authentication logic, time-based quiz state updater,.....

The **css** folder manages the application's styling, including colors and fonts.

Finally, the **utils** folder can be used for a variety of things, such as reusable library instances, some dummy data, or reusable utility functions.

3.5.2 Backend

Figure 3.13 shows the backend architecture:

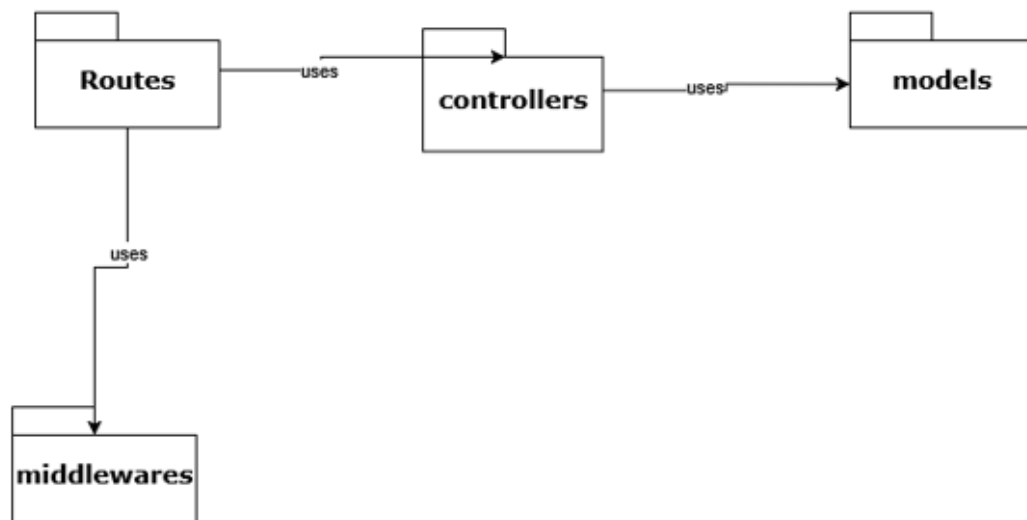


Figure 3.13: Backend architecture diagram

The **models** are responsible for interacting with the database — reading, writing, updating, and deleting records.

The **controllers** folder handle the logic of the application. They receive input from routes, interact with models, and return a response (JSON, HTML, etc.).

The **routes** are responsible for mapping incoming HTTP requests (URLs) to the appropriate controller functions.

The **middlewares** folder provides reusable functions for handling requests, authentication, validation, and error handling across various routes.

3.6 User Interface design

The user interface of the application is role-based and adapts dynamically depending on the actor interacting with the system—Guest, Student, Lecturer, or Headmaster. This design ensures that each user only accesses functionalities relevant to their role, improving usability and maintaining data integrity.

3.6.1 Sidebar navigation

The sidebar serves as the primary navigation panel and is context-aware. For Students, the sidebar displays navigation options such as View Exam, Take Exam, Ranking, and History. In contrast, Lecturers are presented with administrative tools like Manage Exams, View Results, Manage Question Bank, and Manage Student List. The Headmaster, on the other hand, gains access to Manage Class. This dynamic sidebar will ensure a more secure and relevant experience per role.

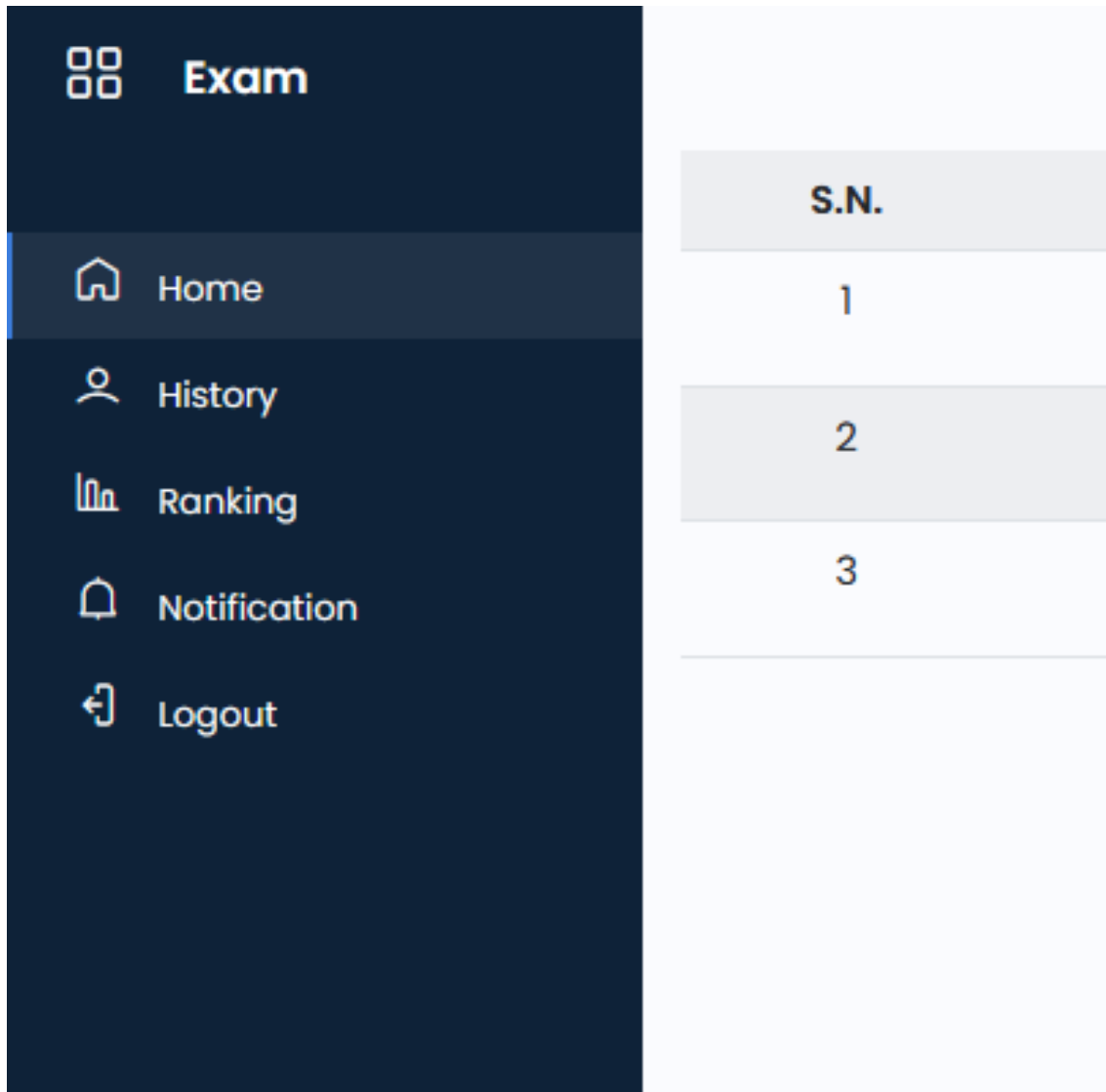


Figure 3.14: Sidebar screen

3.6.2 Main content area

The main content area is tightly coupled with the user's role and the use cases available to them. For instance, when a Student selects "Home" from the sidebar, the main area loads a table showing available exams with contextual buttons for each exam. For Lecturers and Headmasters, selecting "Manage Exam" brings up

tools for creating, editing, or removing exams. The content view switches dynamically based on user actions and presents only the necessary components, often in tabular format with supporting action buttons.

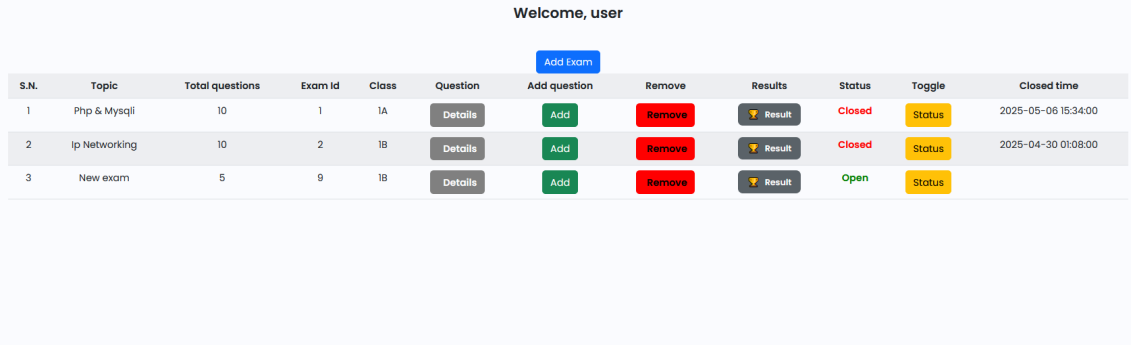


Figure 3.15: Main content screen

3.6.3 Buttons

Buttons in the system are multifunctional and vary by context. A common usage includes navigating to different pages or modules—for example, clicking "Take" on an open exam will route the user to the exam interface. Additionally, buttons serve administrative functions like Create, Edit, or Delete, often triggering modals for input. These modals allow for adding new exams, questions, or managing user details without leaving the current view. Buttons are color-coded for easier indication, and in some case, enabled/disabled based on status. For example, "Take" is disabled if the exam is closed, and vice versa.

3.6.4 Responsiveness and Role-Driven Adaptability

The interface is designed to be responsive, ensuring compatibility across various devices. More importantly, the app’s architecture enforces role-driven adaptability, meaning all interface elements (sidebar, content, actions) change contextually based on the logged-in actor. This keeps the user experience both secure and focused, ensuring that each actor is presented with only their relevant features, aligning tightly with the system’s use case model.

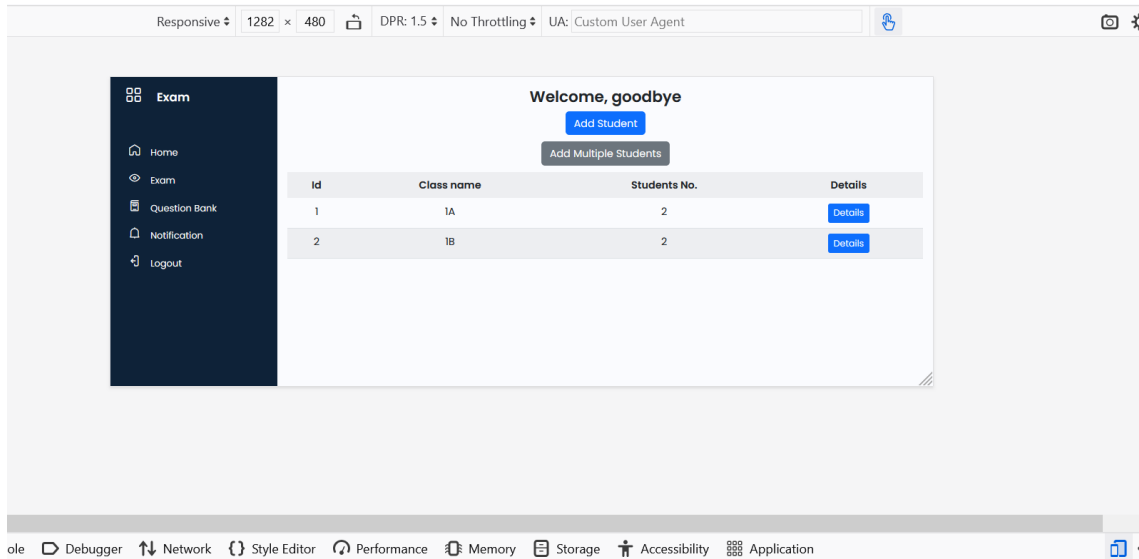


Figure 3.16: Responsive interface

3.7 Database design

3.7.1 Entity-relationship diagram

Figure 3.17 shows the ER diagram that visually represents the relationships between different entities in the system

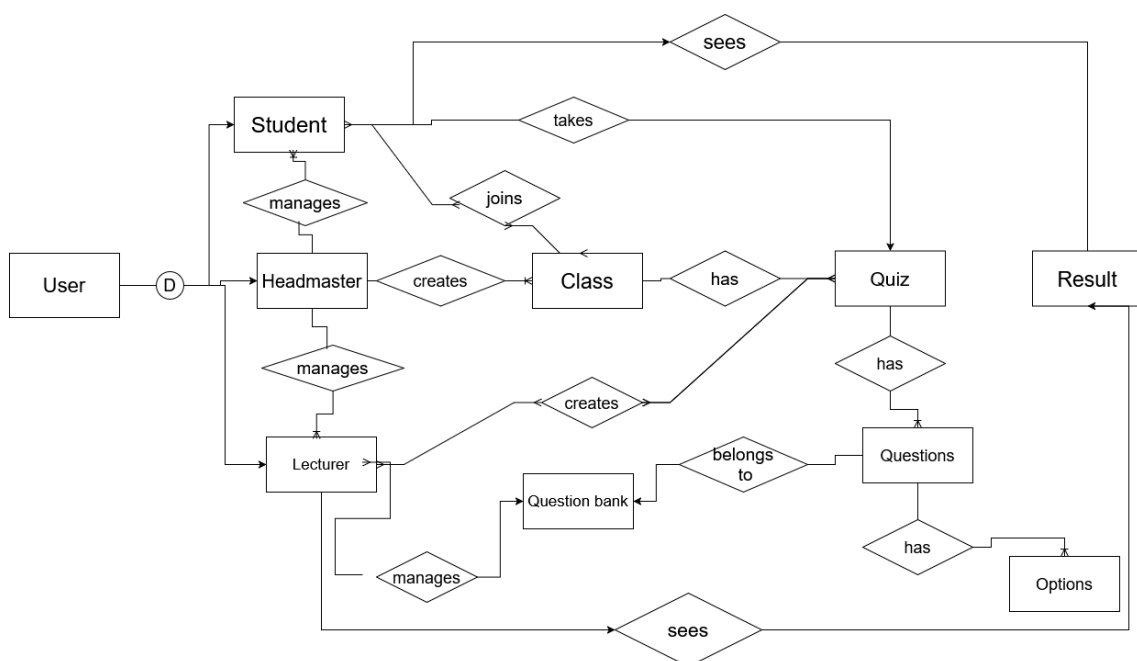


Figure 3.17: Entity-relationship diagram

3.7.2 Database schema

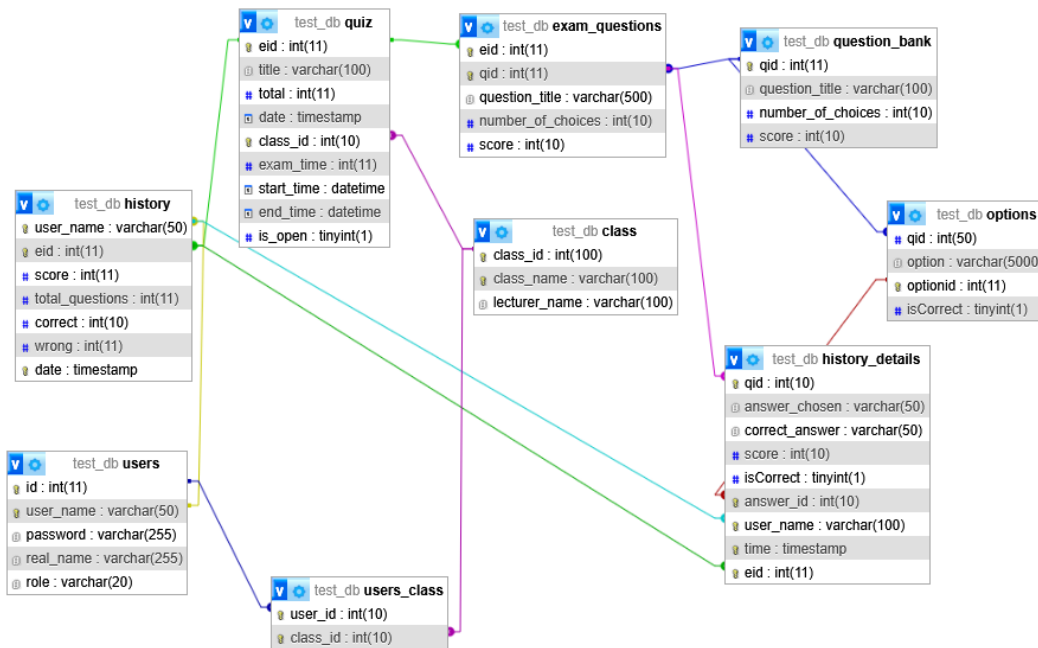


Figure 3.18: Database schema

Attribute	Data type	Description
Id	Int	Unique identifier for user
user_name	String	Username of user
password	String	Encrypted password for authentication
real_name	String	Full name of user
role	String	Role of user

Table 3.6: Detail of the entity "User"

Attribute	Data type	Description
class_id	Int	Unique identifier for class
class_name	String	Name of class
lecturer_name	String	Full name of lecturer

Table 3.7: Detail of the entity "Class"

Attribute	Data type	Description
eid	Int	Unique identifier for the exam
title	String	Title of the exam
total	Int	Total number of questions
date	Timestamp	Date the quiz was created
class_id	Int	Class id that is assigned the exam
exam_time	Int	Duration of the exam in minutes
start_time	Datetime	Scheduled exam start time
end_time	Datetime	Scheduled exam end time
is_open	Tinyint(1)	Exam availability status (1=open, 0=closed)

Table 3.8: Detail of the entity "Quiz"

Attribute	Data type	Description
user_name	String	Username of the student attempted quiz
eid	Int	Associated exam ID
score	Int	Total score obtained
total_questions	Int	Total questions in the quiz
correct	Int	Number of correct answers
wrong	Int	Number of wrong answers
date	Timestamp	Date student attempted quiz

Table 3.9: Detail of the entity "History"

Attribute	Data type	Description
qid	Int	Question ID that the answer belongs to
answer_chosen	String	Option selected by student
correct_answer	String	Correct option for the question
score	Int	Score awarded for the question
isCorrect	Boolean	Correct or wrong answer indicator
ans_id	Int	Unique identifier for the question answer
user_name	String	Username of the student
time	Timestamp	Time of answer submission
eid	Int	Associated exam ID that the answer belongs to

Table 3.10: Detail of the entity "History_Details"

Attribute	Data type	Description
eid	Int	Exam ID that the question belongs to
qid	Int	Id of the exam question
question_title	String	Title of the question
number_of_choices	Int	Number of answer options
score	Int	Score assigned to the question

Table 3.11: Detail of the entity "Exam_Questions"

Attribute	Data type	Description
qid	int	Id of question bank question
question_title	String	Title of the question
number_of_choices	Int	Number of choices in the question
score	Int	The score of the question

Table 3.12: Detail of the entity "Question bank"

Attribute	Data type	Description
qid	Int	Id of the question that the option belongs to
option	String	Content of the question's option
optionid	Int	Id of the option
isCorrect	boolean	Indicator of correct or wrong option

Table 3.13: Detail of the entity "Options"

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

4.1 Achieved results

These contents below are the result of the design and development process of my project. The system has all developed functionalities for students, lecturers and headmasters: When users access the website through a web browser on a PC or laptop, they will be directed to the system's homepage. From there, they can perform login function:

Exam website home page interface

When the user clicks the "login" button, the website will navigate and open a new window asking for the role of user before login:

Role interface

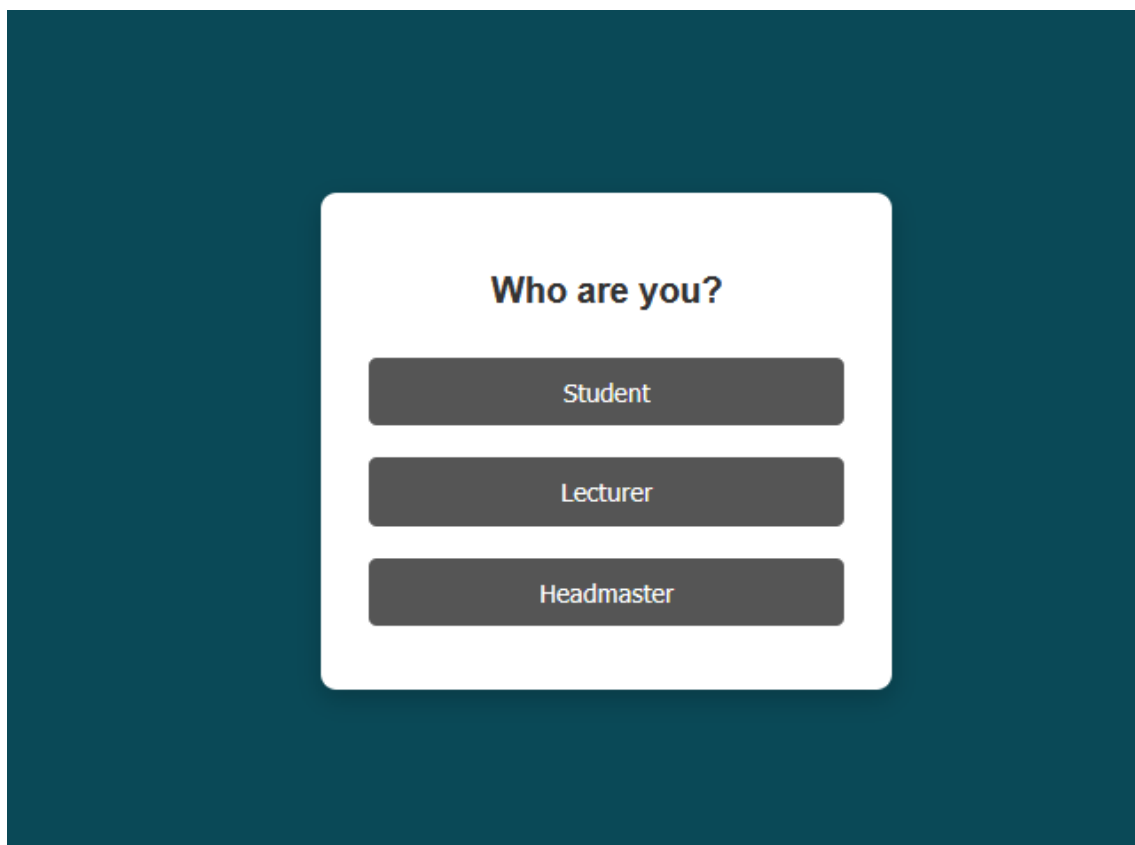


Figure 4.1: Exam website role interface

When the user clicks on the desired role, the website will navigate and open a new window containing the login interface, depend on the chosen role:

Login interface

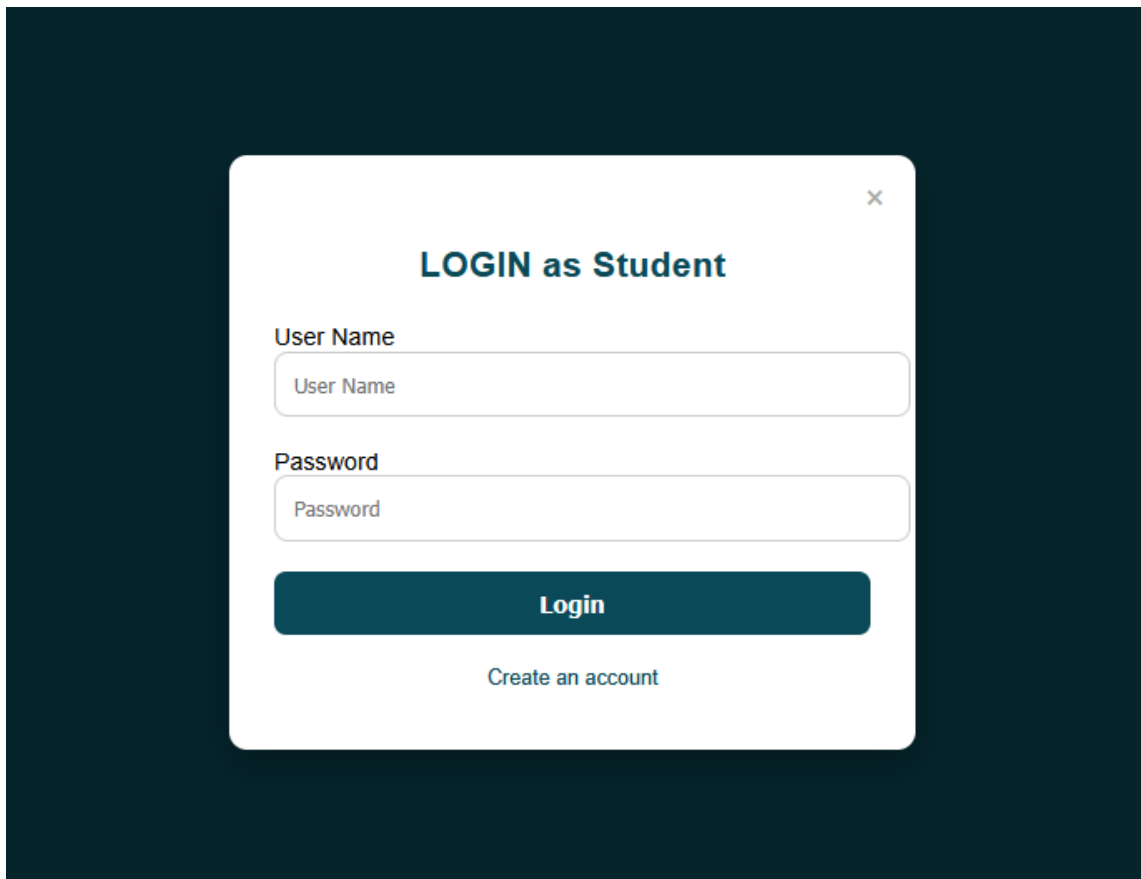
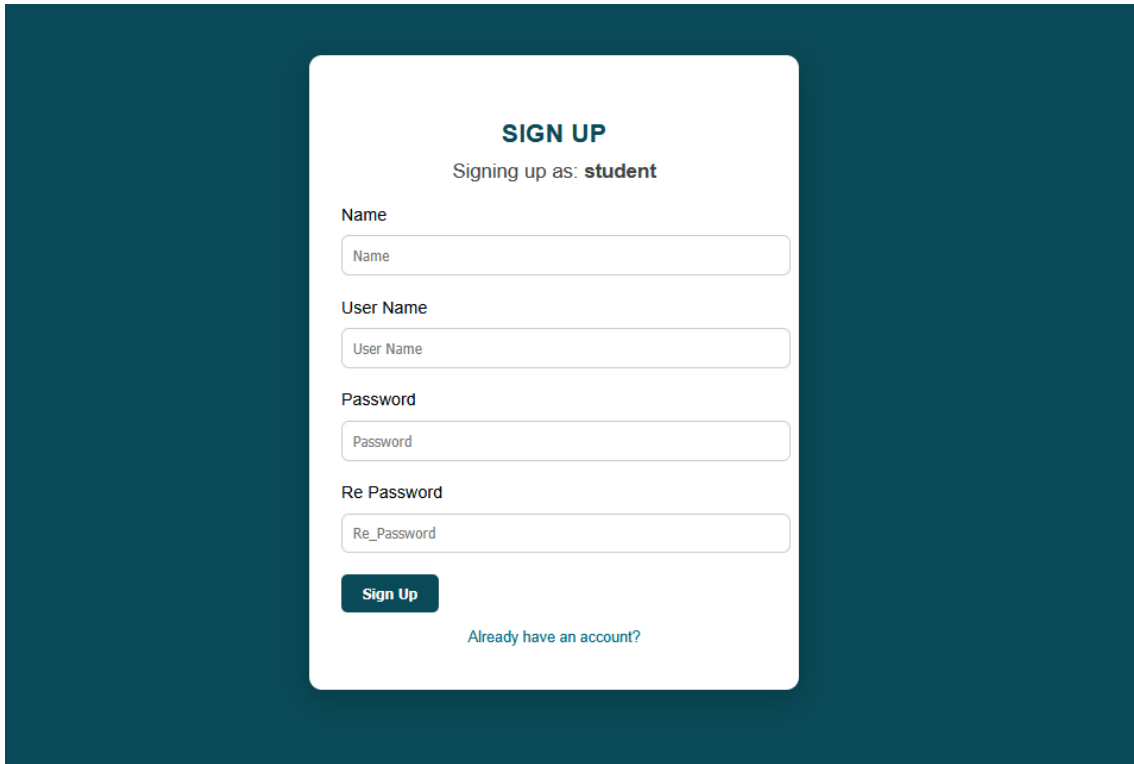


Figure 4.2: Login interface for students

If the user has not registered an account, depend on the role, clicking the "register" button will navigate the website and open a new window containing the registration interface for respective role user want to create account

Registration interface



SIGN UP

Signing up as: **student**

Name

User Name

Password

Re Password

Sign Up

[Already have an account?](#)

Figure 4.3: Registration interface for students

4.1.1 Student's interfaces

Upon clicking Login after inputting necessary information, students are directed to the exam homepage where they are ready to take exam depend on class they are in:

Home page interface

Exam

Home

History

Ranking

Notification

Logout

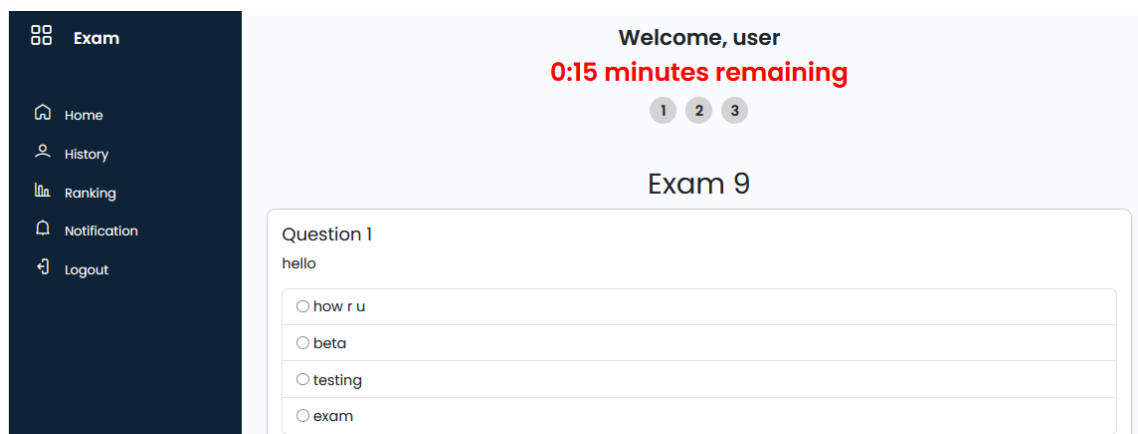
Welcome, hello

S.N.	Title	Total questions	Exam Id	Class	Status	Take Exam
1	Php & Mysql	10	1	1A	Closed	Take
2	Ip Networking	10	2	1B	Closed	Take
3	New exam	5	9	1B	Open	Take

Figure 4.4: Home page interface for students

After chosen the desired exam to take, students will be asked for confirmation, before directing to the "Take exam" interface:

Take exam interface

**Figure 4.5:** Take exam interface for students

Upon finishing the exam, students are directed to the history page, where they can click on the details and see which answer of them are correct or wrong. Every time students take same exam, a new history will overwrite the old history to avoid the data complexity:

History interface

S.N.	Quiz	Question Solved	Right	Wrong	Score	Details
1	New exam	0	0	0	0	Details
2	Php & Mysql	4	1	3	1	Details

Figure 4.6: History interface

History detail interface

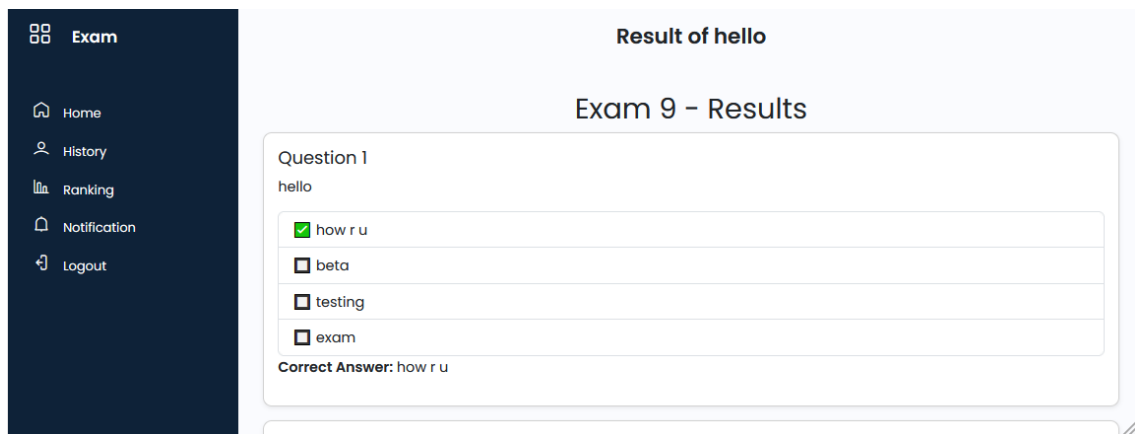


Figure 4.7: History detail interface

After seeing history, students can also access Ranking table, by inputting Exam title and class name to see their current position. Just like History, once students attempt their exam for the 2nd time, new Ranking will overwrite the old Ranking:

Ranking interface

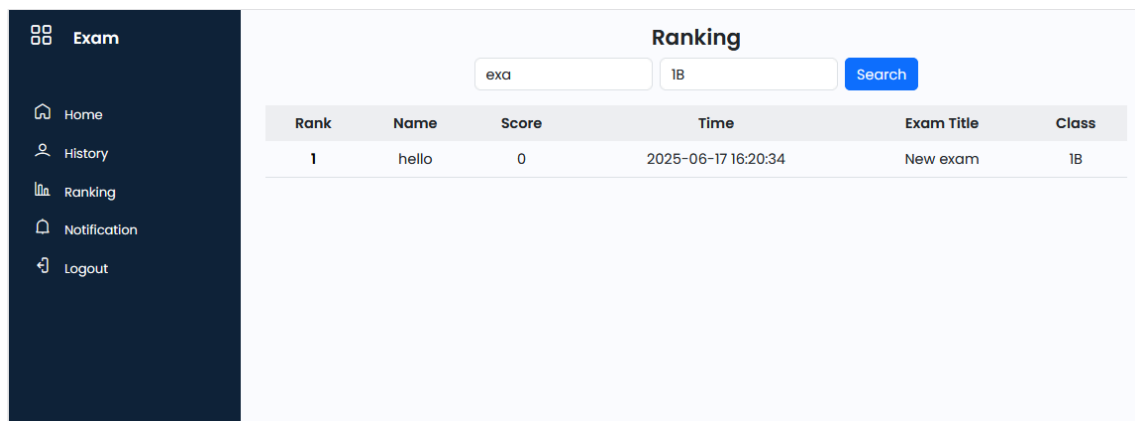


Figure 4.8: Ranking interface

To be able to take part in an exam, students need to enroll a class beforehand, and the Notification interface is meant to notify students of their status, whether they are being added into class or not:

Notification interface

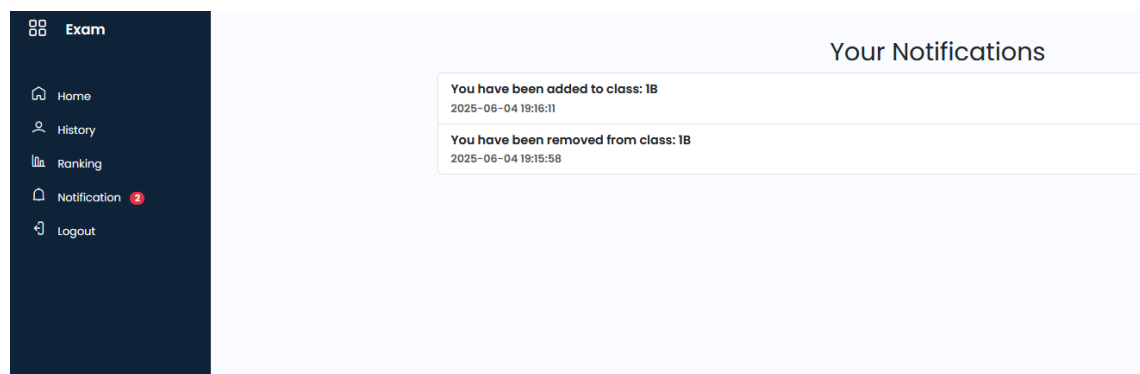


Figure 4.9: Notification interface

4.1.2 Lecturer's interfaces

Upon clicking Login after inputting necessary information about the lecturer's account, lecturers are directed to the homepage where they are ready to view class details, and add students into class:

Lecturer's home page interface

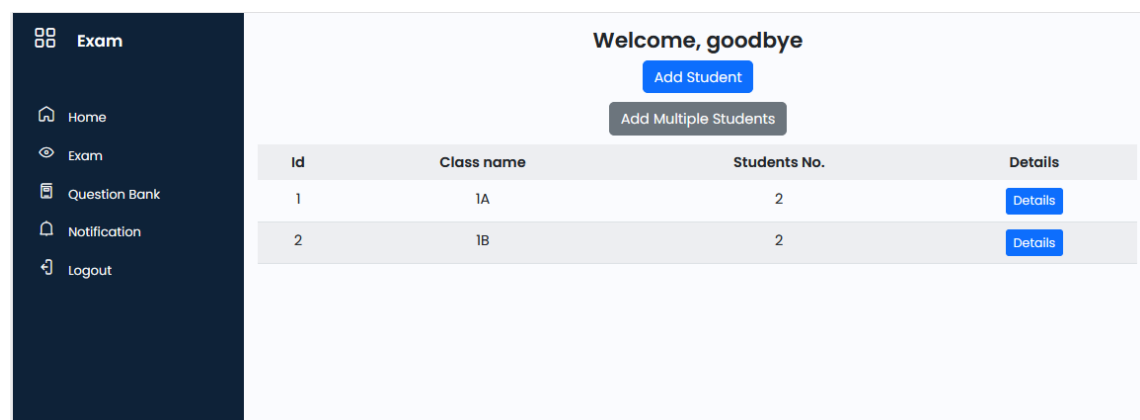


Figure 4.10: Home page interface for Lecturers

To start managing exams, Lecturers can click "Exam" button at the sidebar, and they will be directed to manage exam interface. From there, aside from basic functions, they can add new question to exam, see the exam results that each student had taken, or manipulate exam status:

Manage exam interface

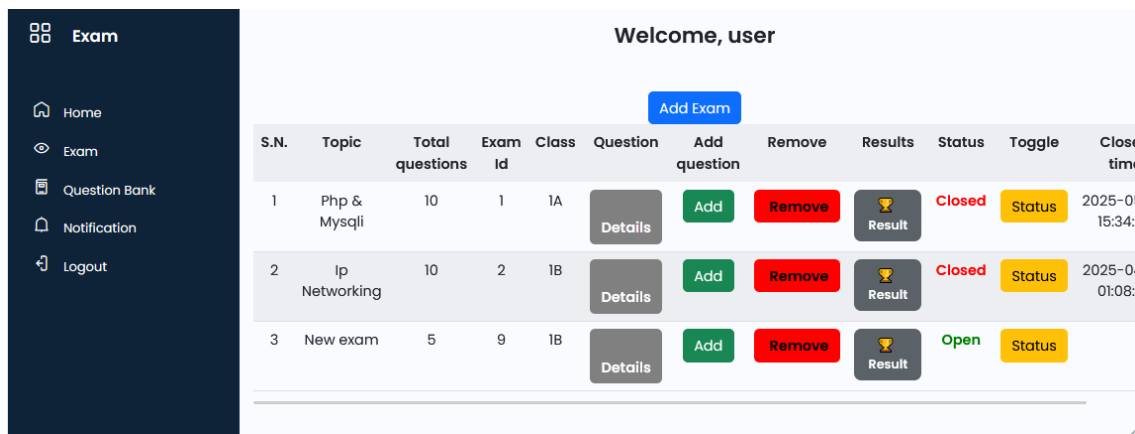


Figure 4.11: Manage exam interface

When lecturers want to manage question bank, they need to access the question bank interface, in where they can add/delete new question, add type to question,..... Question bank questions are divided by type for lecturers to have an easier time to see what question from bank to be added into exam:

Question bank interface

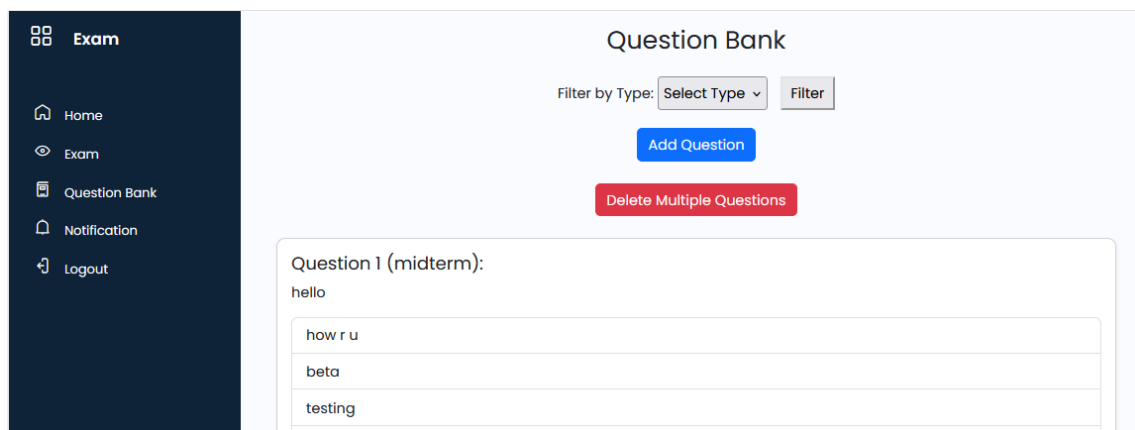


Figure 4.12: Question bank interface

Just like students, lecturers also has their notification interface where they get notified of being added or removed out of class by headmaster:

Notification interface

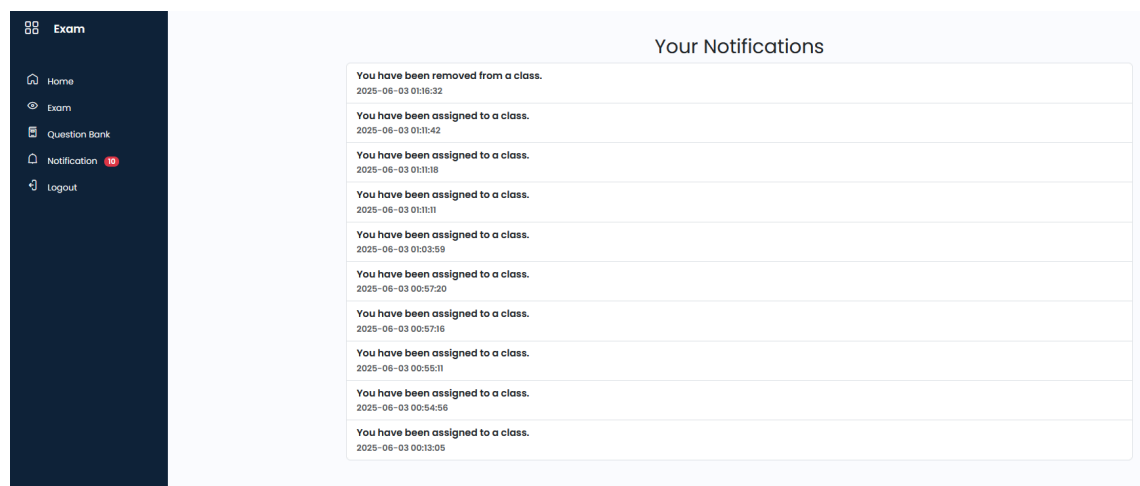


Figure 4.13: Lecturer notification interface

4.1.3 Testing

The following main function will be tested and the results are shown in the following tables:

Table 4.1: Add question into bank Test case

No	Purpose	Process	Expected result	Reality result	Number of Tests	Passed Tests	Pass Rate
TC001	Check adding question into question bank (Lecturer)	- Click on the "Add question" button. - Input info - Click the "Submit" button to submit the information.	Display the uploaded question in the bank and display success notification	Display the uploaded question in the bank and display success notification	10	10	100%

Table 4.2: Add student into class Test case

No	Purpose	Process	Expected result	Reality result	Number of Tests	Passed Tests	Pass Rate
TC002	Check adding multiple student into class (Lecturer)	• Click on the "Add multiple student" button. • Insert the Csv file required • Click the "Upload and add users" button to submit the information.	Display the uploaded students and display success notification	Display the uploaded students and display success notification	10	10	100%

Table 4.3: Add student into class Test case

No	Purpose	Process	Expected result	Reality result	Number of Tests	Passed Tests	Pass Rate
TC003	Check taking exam (Student)	• Click on the "Take" button. • Confirm the choice and start taking exam • Answer exam questions. • Submit the answers	Exam results are displayed in the history section	Exam results are displayed in the history section	10	10	100%

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

This thesis set out to design and implement a lightweight, modular online examination management system tailored for small to mid-sized educational institutions facing limitations with existing solutions. By focusing on core functionalities such as exam creation, class-based assignment, question bank management, exam delivery, and basic automated performance tracking, the system avoids the complexity and overhead often associated with commercial or overly generalized platforms.

Using PHP and MySQL for the backend and standard web technologies (HTML, CSS, JavaScript, and Bootstrap) for the frontend, the developed system delivers a clean, responsive interface for both students and lecturers. Crucially, the system meets its design goal of operating without reliance on external dependencies, enabling ease of deployment in constrained or offline-capable environments. The use case design supports clearly defined user roles (guest, student, lecturer, headmaster), ensuring that access control and system interactions remain intuitive and secure. Testing confirmed the platform's ability to handle key workflows, including exam visibility scheduling, student enrollment via CSV, class-based exam filtering, and performance ranking visualization.

The system successfully implements both exam-taking and automated grading, allowing students to answer questions online and receive immediate evaluation based on predefined correct answers. This is achieved through a modular backend and a scalable relational schema that tracks exam attempts, individual question responses, and overall performance metrics. With features like a centralized question bank, dynamic exam-question linkage, and automatic ranking generation, the project not only meets its intended goals but also lays a solid groundwork for future enhancements such as, question randomization, and analytics dashboards,...

Over the course of this project, I applied and significantly deepened my understanding of full-stack web development, starting with the foundational skills of building responsive interfaces using HTML, CSS, JavaScript, and Bootstrap. As the system evolved, I moved into more complex backend development with PHP and MySQL — designing and managing relational database structures, implementing secure login sessions, and building logic for user roles like lecturers, students, and headmasters. I successfully developed key features such as class-based exam filtering, timed exam visibility,..... Each of these required integrating both frontend and backend logic, as well as writing code that balanced usability, efficiency, and

scalability. The experience of building these interconnected components taught me how even seemingly small features can require deep coordination across the entire tech stack.

Despite the core features being implemented, several important areas remain as limitations in the current version. The system lacks support for essay-style questions that require manual grading, real-time updates through AJAX or reactive frameworks..... Features such as mobile-first design, analytics dashboards, or Progressive Web App support were not included due to time constraints and technical trade-offs.

In addition, I learned that writing scalable and maintainable code is not just about functionality, but about planning for future features without adding unnecessary complexity to the current system. More importantly, this project taught me how to balance ideal goals with practical constraints — a lesson that mirrors the reality of software development.

5.2 Future work

To further enhance the functionality of the online exam system, I propose several additional features and improvements. These enhancements would elevate the system's functionality and robustness.

For lecturers, a key feature to consider is the incorporation of data analytics tools would greatly enhance their ability to assess student progress and improve exam design. Features such as performance trend analysis, class-wide statistics, question difficulty ratings, and individual score distributions would allow for deeper insights into learning outcomes. Visualizations like heatmaps or charts could highlight which topics students struggle with most, enabling more targeted teaching strategies and question revisions.

For students, their experience could also benefit from adopting AJAX or reactive front-end frameworks such as Vue.js or React. This would reduce full-page reloads in data-heavy views like student lists or question banks, making the platform more responsive and modern.

Another quality-of-life improvement worth implementing is a "Continue Exam" feature, allowing students to resume an in-progress exam session if they are unexpectedly logged out or disconnected. This functionality would require session persistence and server-side tracking of exam state, including the questions already answered and remaining time. By enabling students to continue from where they left off, the system would become more resilient to interruptions and offer a smoother,

more forgiving user experience — especially important in regions with unstable internet connections.

By incorporating these improvements, the online exam system could evolve from a lightweight testing platform into a comprehensive educational tool that supports both student evaluation and teaching strategies. These enhancements would not only improve usability and academic integrity but also empower educators with actionable insights — making the system a valuable asset for data-driven teaching in both traditional and remote learning environments.

REFERENCE

[1] Moodle Project, *Moodle - Open-source learning platform*. [Online]. Available: <https://moodle.org> (visited on 06/10/2025).

[2] Common Sense Education, *Teachers' Essential Guide to Google Classroom*. [Online]. Available: <https://www.commonsense.org/education/articles/teachers-essential-guide-to-google-classroom> (visited on 06/14/2025).